

Baochip–STM32H743 Secure Boot Architecture

1. Objective

The objective is to establish a secure boot chain between a Baochip security controller and an STM32H743 flight controller.

The central security requirement is:

The H743 may receive arbitrary firmware, but it must execute firmware only after that firmware has been authenticated by a trusted bootloader.

The architecture separates the responsibilities of the Baochip and the H743:

- **Baochip** establishes and protects the trust boundary for the H743 bootloader.
 - **H743 secure bootloader** authenticates the flight-controller firmware before execution.
 - **H743 firmware** runs only if the bootloader considers it authentic.
-

2. Components

Baochip

Baochip acts as the external root of trust for the H743 bootloader.

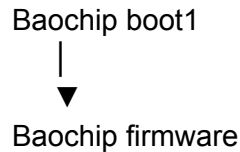
Its relevant responsibilities are:

- Maintain the trusted firmware-signing/public-key information.
- Hold the H743 in reset when necessary.
- Authenticate the H743 secure bootloader.
- Release the H743 from reset only after successful authentication.
- Optionally provide a secure communication/update channel to the H743.

Baochip itself has its own secure boot chain:

Baochip boot0





`boot0` is the initial trusted component that authenticates `boot1`.

3. H743 Secure Bootloader

The H743 secure bootloader is firmware stored in the H743's Flash memory.

It is distinct from the STM32 factory ROM bootloader.

STM32 factory ROM bootloader

The H743 contains a factory-programmed ROM/system-memory bootloader supplied by ST.

It is used as a programming/recovery mechanism and is not the secure bootloader described in this architecture.

H743 secure bootloader

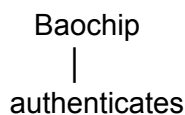
The secure bootloader is application firmware that is provisioned into H743 Flash.

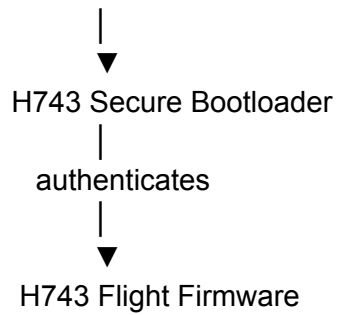
Its responsibility is:

1. Locate the installed flight firmware.
 2. Authenticate its signature.
 3. Reject unauthenticated firmware.
 4. Execute the firmware only if authentication succeeds.
-

4. Trust Chain

The proposed trust chain is





Therefore:

Baochip → Bootloader → Flight Firmware

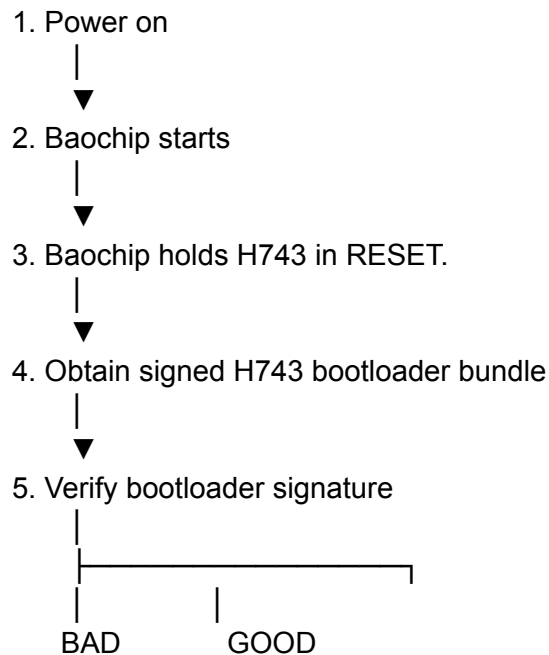
The H743 flight firmware does not become trusted merely because it was written into Flash.

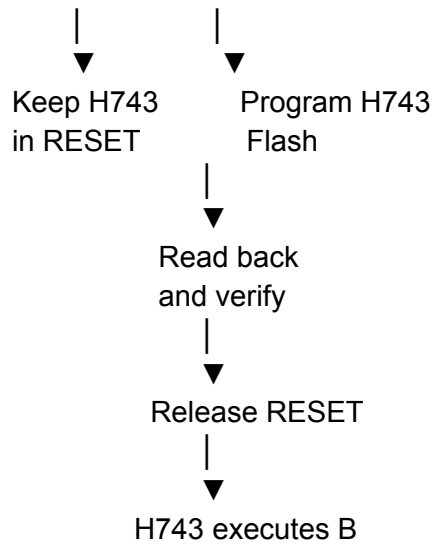
It becomes executable only after the secure bootloader validates it.

5. Initial Provisioning

During initial provisioning, Baochip keeps the H743 in reset.

The sequence is





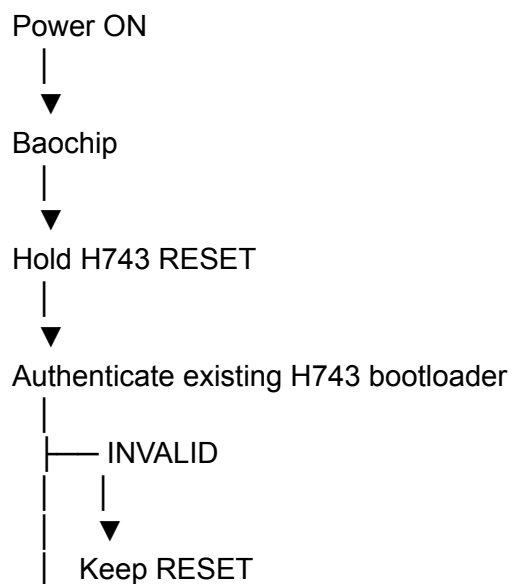
If the bootloader signature is invalid, Baochip must not release the H743 from reset.

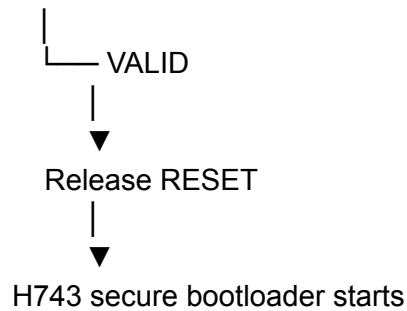
6. Subsequent Power-On

The bootloader does not need to be erased and rewritten on every power cycle.

Flash is non-volatile, so the provisioned bootloader remains after power is removed.

A normal boot therefore looks like:





Erase/write operations are required primarily during:

- Initial provisioning
- Bootloader updates
- Recovery/replacement

They are not inherently required on every power-on.

7. Firmware Installation and Updates

A key property of this architecture is that **writing firmware and executing firmware are separate operations**.

Someone may potentially write arbitrary data into the H743's application area.

That does not make the firmware trusted.

For example:

H743 Flash

Secure Bootloader B	← protected
Application area	
C / C_malicious / anything	← may be written

On startup, the secure bootloader decides whether the contents of the application area are executable.

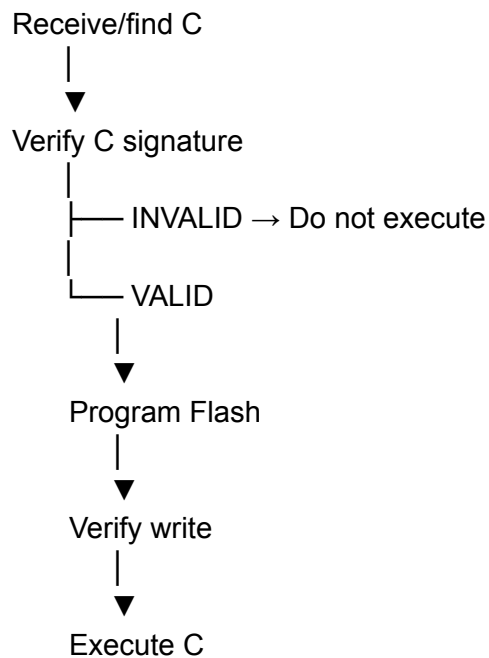
8. Installing Flight Firmware C

Suppose firmware C is the desired flight controller firmware.

The authorized signing process produces:

C
+
signature
+
optional metadata/version information

The H743 secure bootloader then performs:



The bootloader can receive the firmware through an authorized update mechanism, potentially involving Baochip.

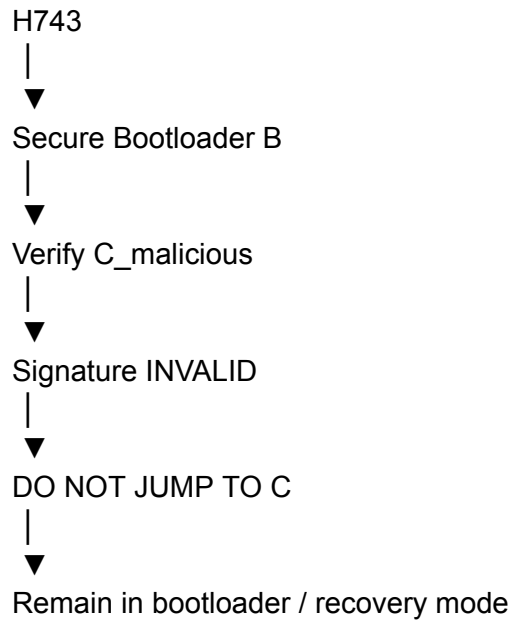
9. Handling Malicious Firmware

Suppose an attacker flashes:

C_malicious

into the H743 application area.

The next boot becomes:



Therefore:

The ability to write Flash does not imply the ability to execute arbitrary firmware.

This is the fundamental security property of the architecture.

10. Protecting the Secure Bootloader

The architecture depends critically on the integrity of bootloader B.

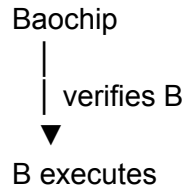
If an attacker can replace B with:

B_malicious

then the attacker could create a bootloader that simply accepts any application.

Therefore the bootloader itself must be protected.

The proposed protection is:



and the H743 should additionally use available MCU Flash/security protections to prevent unauthorized modification of the bootloader region.

The precise combination of Baochip verification and H743 hardware protection should be determined during implementation.

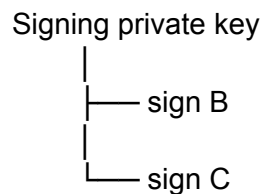
11. Key Hierarchy

The architecture requires a clear distinction between:

Signing private key

Held by the authorized firmware-signing authority.

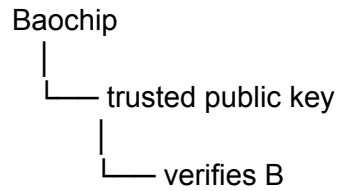
Used to create signatures for trusted firmware.



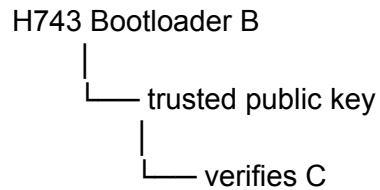
Verification public key

Stored in the component that needs to authenticate firmware.

For example:



and:



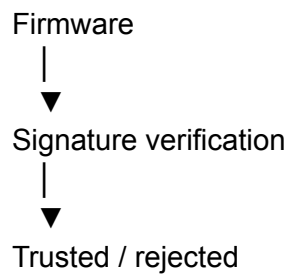
The exact relationship between the HoloDi keys and Baochip's provisioned keys must be confirmed from the actual Baochip/HoloDi implementation rather than assumed.

12. Pairing vs. Secure Boot

Pairing and secure boot should be treated as separate security mechanisms.

Secure boot answers:

"Is this firmware authentic?"



Pairing answers questions such as:

"Is this the authorized Baochip/FC relationship?"

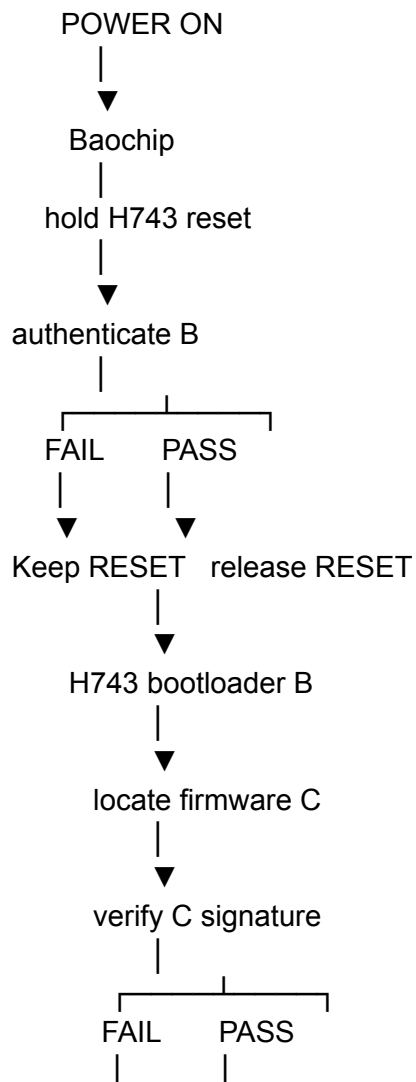
and potentially:

- Which Baochip is authorized to communicate with this H743?
- Which H743 is associated with this Baochip?
- Should firmware updates be permitted?
- Should communications be encrypted?
- How should session keys be established?
- How is freshness/replay protection provided?

Therefore pairing can sit alongside the secure-boot mechanism rather than replacing it.

13. Proposed Normal Boot Sequence

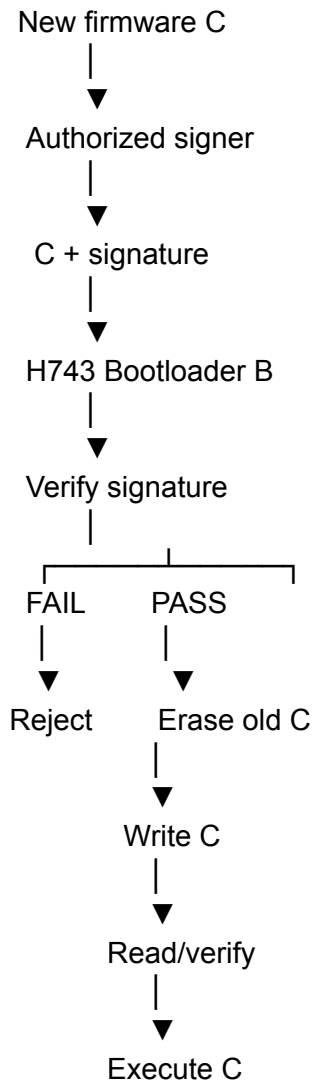
The complete normal boot sequence is:



▼ ▼
stay in B execute C

14. Proposed Update Sequence

For an application update:



Baochip may participate in transporting or authorizing the update, but the H743 bootloader remains responsible for deciding whether the application is authentic.

15. Security Properties

The architecture aims to provide the following properties:

Property 1 — Untrusted firmware cannot execute

An attacker may modify the application Flash contents, but the secure bootloader refuses to execute firmware with an invalid signature.

Property 2 — Untrusted bootloader cannot execute

Baochip verifies the H743 bootloader before releasing the H743 from reset.

Property 3 — Power cycling does not reinstall firmware

Provisioned firmware remains in non-volatile Flash.

Property 4 — Firmware updates require authorization

Only firmware carrying a valid signature from the trusted signing authority is accepted.

Property 5 — Baochip is outside the H743 trust boundary

The H743 bootloader cannot simply declare itself trusted. Baochip provides the initial external trust decision for the bootloader.

16. Hardware Validation Plan

The architecture can be implemented incrementally.

M1 — H743 communication

Demonstrate communication with the STM32 factory ROM bootloader:

Baochip → UART → H743 ROM bootloader

0x7F

↓

0x79 ACK

Then issue `GET_ID` and confirm the expected H743 device ID.

M2 — Reset control

Demonstrate:

Baochip GPIO → H743 NRST

Baochip should be able to:

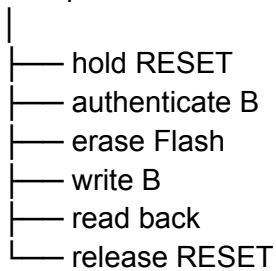
`RESET_HOLD`

`RESET_RELEASE`

M3 — Bootloader provisioning

Demonstrate:

Baochip



M4 — Application authentication

Install:

B + C

and demonstrate:

Valid C → execute

Tampered C → reject

Unsigned C → reject

M5 — Bootloader attack

Attempt to replace B with:

B_malicious

and demonstrate:

Baochip → rejects B

H743 → remains in RESET

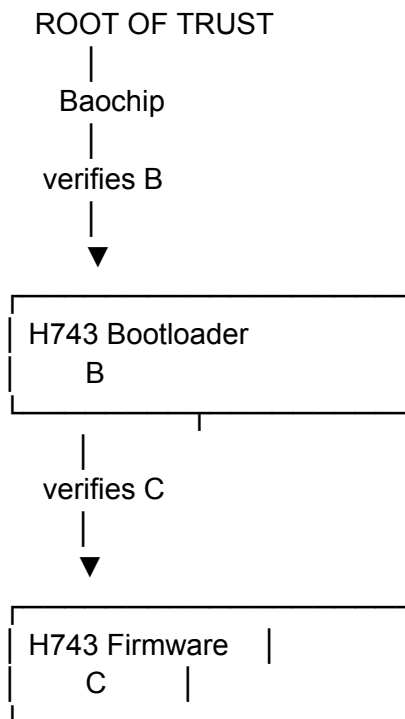
This is the strongest demonstration of the external root-of-trust architecture.

17. Core Design Principle

The architecture can be summarized in one sentence:

Baochip decides whether the H743 bootloader is trusted; the trusted H743 bootloader decides whether the flight firmware is trusted.

Or visually:



The fundamental rule is:

Flashability does not imply executability. Trust is established at the point where code is allowed to execute.